

BDA Lab 1: WordCount using Hadoop MapReduce

Simple lab-manual steps with a basic Java program. The Java code is included mainly for writing and understanding the MapReduce structure.

1. Setup Docker and Hadoop

Run these commands in Windows PowerShell:

```
cd C:\bda-labs
mkdir lab1-wordcount
cd lab1-wordcount
docker run -dit --name hadoop apache/hadoop:3.3.6 bash
docker exec -it hadoop bash
```

2. Create the Java program

Inside the Hadoop container, create a file named WordCount.java. You may use nano:

```
nano WordCount.java
```

Basic Java code for writing:

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            String[] words = value.toString().split("\\s+");

            for (String w : words) {
                word.set(w);
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {

        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values,
            Context context)
            throws IOException, InterruptedException {

            int sum = 0;

            for (IntWritable value : values) {
                sum += value.get();
            }

            result.set(sum);
            context.write(key, result);
        }
    }
}
```

```

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "word count");

        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
        job.setCombinerClass(IntSumReducer.class);
        job.setReducerClass(IntSumReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

Save the file in nano using: Ctrl + O, Enter, Ctrl + X.

3. Create input files

```
mkdir input
```

```
echo "the quick brown fox the lazy dog the fox" > input/file1.txt
```

```
echo "a quick dog and a brown fox jumped" > input/file2.txt
```

4. Compile and create the JAR file

Run these commands inside the container. The classpath command allows Java to find Hadoop libraries. The JAR command includes the main class and inner classes.

```
export HADOOP_CLASSPATH=$(hadoop classpath)
```

```
mkdir classes
```

```
javac -classpath $HADOOP_CLASSPATH -d classes WordCount.java
```

```
jar -cvf wordcount.jar -C classes .
```

5. Run the MapReduce program

```
hadoop jar wordcount.jar WordCount input output
```

6. View the output

```
cat output/part-r-000000
```

Expected output:

```

a          1
and        1
brown     2
dog        2
fox        3
jumped    1
lazy       1
quick     2
the        3

```

7. Run the program again

Hadoop will not overwrite an existing output folder. Remove it before running again:

```
rm -r output
```

```
hadoop jar wordcount.jar WordCount input output
```

```
cat output/part-r-00000
```

8. Exit and clean up

```
exit
```

Back in Windows PowerShell:

```
docker rm -f hadoop
```

Quick command list

```
docker exec -it hadoop bash
export HADOOP_CLASSPATH=$(hadoop classpath)
mkdir classes
javac -classpath $HADOOP_CLASSPATH -d classes WordCount.java
jar -cvf wordcount.jar -C classes .
hadoop jar wordcount.jar WordCount input output
cat output/part-r-00000
```